

Entropy: Measuring Impurity

In decision tree learning, the algorithm needs a way to measure the "purity" of a set of data.

- A **pure** set is one where all examples belong to the same class (e.g., 100% "Cat" or 100% "Not Cat").
- An **impure** set is a mix of classes.

Entropy, denoted as $H(p_1)$, is a mathematical function used to measure **impurity**.

Understanding Entropy with Examples

Let p_1 be the fraction of positive examples ($y = 1$, e.g., "Cat") in a set.

- **Maximum Impurity (Entropy = 1):**
 - This occurs when the data is a perfect 50/50 split.
 - *Example:* 3 Cats, 3 Dogs $\rightarrow p_1 = 0.5 \rightarrow H(p_1) = 1$.
 - **No Impurity (Entropy = 0):**
 - This occurs when the data is 100% pure (all one class).
 - *Example 1:* 6 Cats, 0 Dogs $\rightarrow p_1 = 1.0 \rightarrow H(p_1) = 0$.
 - *Example 2:* 0 Cats, 6 Dogs $\rightarrow p_1 = 0.0 \rightarrow H(p_1) = 0$.
 - **Some Impurity ($0 < \text{Entropy} < 1$):**
 - This occurs with any other mixed set.
 - *Example 1:* 5 Cats, 1 Dog $\rightarrow p_1 \approx 0.83 \rightarrow H(p_1) \approx 0.65$.
 - *Example 2:* 2 Cats, 4 Dogs $\rightarrow p_1 \approx 0.33 \rightarrow H(p_1) \approx 0.92$.
 - Note that the 2/4 split ($p_1 = 0.33$) is *more impure* (closer to 0.5) than the 5/1 split ($p_1 = 0.83$), so its entropy is higher.
-

The Entropy Formula

Let p_1 be the fraction of positive examples and $p_0 = 1 - p_1$ be the fraction of negative examples.

The formula for entropy is:

$$H(p_1) = -p_1 \log_2(p_1) - p_0 \log_2(p_0)$$

or equivalently:

$$H(p_1) = -p_1 \log_2(p_1) - (1 - p_1) \log_2(1 - p_1)$$

Key Conventions:

- **Log Base 2 ($\log_2$):** This is used by convention so that the maximum impurity (at $p_1 = 0.5$) is exactly **1**.
 - **Special Case ($0 \log_2 0$):** Since $\log_2(0)$ is undefined, we use the convention that $0 \cdot \log_2(0) = 0$. This ensures that pure sets ($p_1 = 0$ or $p_0 = 0$) correctly result in an entropy of 0.
-

Alternatives to Entropy

- Other measures of impurity exist, such as the **Gini criteria** (or Gini impurity).
- The Gini function looks very similar to the entropy curve and serves the same purpose: to quantify the impurity of a node.
- This course will focus on using entropy.

Information Gain: How to Choose the Best Split 🌳

When building a decision tree, the algorithm must decide which feature to use for a split at each node (e.g., split on "Ear Shape," "Face Shape," or "Whiskers").

The best split is the one that **reduces entropy the most**. This reduction in entropy is formally known as **information gain**. The algorithm's goal is to pick the feature that provides the **highest information gain**.

Calculating Information Gain

To find the best feature, we calculate the information gain for *every possible split* and pick the highest one.

The process has 3 steps:

1. Calculate the entropy of the "child" nodes for each possible split.
2. Calculate the weighted average entropy for each split.
3. Calculate the information gain by subtracting the weighted average from the parent's entropy.

Reference: Parent Node Entropy

First, we need the entropy of the node *before* the split (the "parent" node). In our root node example, we have 10 examples (5 cats, 5 dogs).

- $p_1 = 5/10 = 0.5$
 - **Parent Entropy (H_{root}) = 1.0** (maximum impurity)
-

Step 1: Calculate Entropy for Each Potential Split

We'll test all three of our features:

- **Split 1: "Ear Shape"**
 - **Left Branch (Pointy):** 5 examples (4 cats, 1 dog). $p_1 = 4/5 = 0.8$.
 - Entropy = 0.72
 - **Right Branch (Floppy):** 5 examples (1 cat, 4 dogs). $p_1 = 1/5 = 0.2$.
 - Entropy = 0.72
- **Split 2: "Face Shape"**
 - **Left Branch (Round):** 7 examples (4 cats, 3 dogs). $p_1 = 4/7 \approx 0.57$.
 - Entropy = 0.99

- **Right Branch (Not Round):** 3 examples (1 cat, 2 dogs). $p_1 = 1/3 \approx 0.33$.
 - $Entropy = 0.92$
 - **Split 3: "Whiskers"**
 - **Left Branch (Present):** 4 examples (3 cats, 1 dog). $p_1 = 3/4 = 0.75$.
 - $Entropy = 0.81$
 - **Right Branch (Absent):** 6 examples (2 cats, 4 dogs). $p_1 = 2/6 \approx 0.33$.
 - $Entropy = 0.92$
-

Step 2: Calculate the Weighted Average Entropy

To get a single impurity score for each *split*, we calculate a **weighted average** of the child node entropies. The weight is the fraction of examples that went into that branch.

- **Ear Shape (Weighted Avg):**
 - $(\frac{5}{10} \times 0.72) + (\frac{5}{10} \times 0.92) = \mathbf{0.72}$
 - **Face Shape (Weighted Avg):**
 - $(\frac{7}{10} \times 0.99) + (\frac{3}{10} \times 0.92) = 0.693 + 0.276 = \mathbf{0.969}$
 - **Whiskers (Weighted Avg):**
 - $(\frac{4}{10} \times 0.81) + (\frac{6}{10} \times 0.92) = 0.324 + 0.552 = \mathbf{0.876}$
-

Step 3: Calculate Information Gain (Select the Best Split)

Information Gain = (Parent Entropy) - (Weighted Average Entropy of Children)

- **Ear Shape Gain:**
 - $1.0 - 0.72 = \mathbf{0.28}$
- **Face Shape Gain:**
 - $1.0 - 0.969 = \mathbf{0.03}$
- **Whiskers Gain:**
 - $1.0 - 0.876 = \mathbf{0.12}$

Decision: The "Ear Shape" split results in the **highest information gain** (0.28), so the algorithm chooses it as the feature for the root node.

(This process is then repeated at each new node, using only the subset of data at that node.)

Formal Definition of Information Gain

- $H(p_1^{root})$ = Entropy of the parent node
- w^{left} = Fraction of examples going to the left child
- $H(p_1^{left})$ = Entropy of the left child
- w^{right} = Fraction of examples going to the right child
- $H(p_1^{right})$ = Entropy of the right child

$$InformationGain = H(p_1^{root}) - \left[w^{left} H(p_1^{left}) + w^{right} H(p_1^{right}) \right]$$

The Decision Tree Learning Algorithm 🌳

The process of building a decision tree is **recursive**. It starts with the entire dataset at the root node and iteratively splits the data into smaller, purer subsets until a stopping criterion is met.

The Core Algorithm (Iterative Splitting)

1. **Start at the Root Node:** Begin with the complete set of training examples.
2. **Check Stopping Criteria:** At the current node, first check if any **stopping criteria** (see below) have been met.
 - **If YES:** Stop splitting. This node becomes a **leaf node**. Assign it the majority class of the examples it contains (e.g., "Cat" if 4 out of 5 examples are cats).
 - **If NO:** Proceed to step 3.
3. **Find the Best Split:**
 - Calculate the **information gain** for *every* possible feature on the *current* subset of data.
 - Select the feature that provides the **highest information gain**.
4. **Create Branches:**
 - Split the current node's dataset into two new subsets (e.g., a left branch and a right branch) based on the values of the feature you just selected.
5. **Recurse:**
 - Apply this entire process (starting from Step 2) independently to both the left and right branches.

This recursive process (a function that calls itself on smaller subsets) continues until all branches have ended in leaf nodes.

Stopping Criteria (Preventing Overfitting)

If the algorithm continued splitting until every node was 100% pure, the tree would become massive, complex, and would **overfit** the training data. To prevent this, we use **stopping criteria** (also called **pre-pruning**):

- **1. Maximum Purity:** Stop splitting if the node is already 100% pure (i.e., all examples belong to a single class; entropy is 0).
- **2. Maximum Depth:** Stop splitting if the tree reaches a predefined `max_depth` (a hyperparameter, e.g., 10 levels deep).
- **3. Minimum Information Gain:** Stop if the best possible split provides an information gain that is below a small threshold (i.e., the split isn't "worth it").

- **4. Minimum Examples in Node:** Stop if the number of examples in the current node falls below a certain threshold.
-

Hyperparameters and Overfitting

The `max_depth` parameter is a key tool for controlling the bias-variance trade-off in a decision tree:

- A **large `max_depth`** (or no limit) allows the tree to grow very large and complex. It can fit the training data perfectly but is at high risk of **overfitting (high variance)**.
- A **small `max_depth`** forces the tree to be simple and shallow. It is less likely to overfit but is at high risk of **underfitting (high bias)**.

You can use a cross-validation set to find the optimal value for hyperparameters like `max_depth`.

Making Predictions (Inference)

Once the tree is built, making a prediction on a new example is simple:

1. Start at the root node.
2. Follow the branches down the tree based on the example's feature values.
3. The prediction is the class specified by the leaf node you arrive at.

Handling Features with More Than Two Categories

The Problem

So far, our features have been binary (e.g., "Pointy" vs. "Floppy"). What happens if a feature has *more than two* discrete values?

- **Example:** Ear Shape can now be "Pointy", "Floppy", or "Oval".

While you *could* build a tree that splits into three branches, a more common and robust method is to use **one-hot encoding**.

The Solution: One-Hot Encoding encode

One-hot encoding is a technique that transforms a single categorical feature into multiple new **binary (0 or 1) features**.

- **Rule:** If a categorical feature has k possible values, you create k new binary features.

How it Works

Instead of one "Ear Shape" feature, we now create three new features:

1. Is_Pointy
2. Is_Floppy
3. Is_Oval

An example is "encoded" by placing a 1 in the column corresponding to its value and 0s in all other columns.

Example Transformation:

- An animal with "Pointy" ears becomes:
 - Is_Pointy: **1**
 - Is_Floppy: **0**
 - Is_Oval: **0**
- An animal with "Oval" ears becomes:
 - Is_Pointy: **0**
 - Is_Floppy: **0**
 - Is_Oval: **1**

This is called **"one-hot"** because for any given example, exactly *one* of these new features is "hot" (set to 1).

Benefits and Applications

- **For Decision Trees:** This transformation converts the complex categorical feature back into a set of simple binary features. Our decision tree learning algorithm (using information gain) can then use these new 0/1 features without any modification.
- **For Other Models:** One-hot encoding is a standard technique used to prepare categorical data for **any** model that requires numerical inputs, including **neural networks** and **logistic regression**.

Handling Continuous Features

So far, we have only worked with **categorical features** (e.g., "Pointy" vs. "Floppy"). This lesson explains how decision trees can also split on **continuous features** (e.g., any number, like weight).

The Problem

How can a decision tree split a feature like `weight`, which can be any numerical value (e.g., 7.5, 9.1, 13.2)?

The Solution: Thresholding

Instead of splitting on discrete categories, the algorithm splits the data by finding an optimal **threshold**. The decision node's question becomes a binary (True/False) comparison.

- **Example:** Is `weight <= 9`?
 - If **True**, go to the left branch.
 - If **False** (meaning `weight > 9`), go to the right branch.
-

How to Find the Best Threshold?

The learning algorithm must determine the best possible threshold to use. It does this by testing many different values and calculating the **information gain** for each one.

The Process:

1. **Try a Threshold (e.g., `weight <= 8`):**
 - This splits the 10 examples into two groups:
 - **Left Branch:** 2 examples (2 Cats, 0 Dogs)
 - **Right Branch:** 8 examples (3 Cats, 5 Dogs)
 - The algorithm calculates the weighted average entropy and finds the **Information Gain = 0.24**.
2. **Try Another Threshold (e.g., `weight <= 9`):**
 - This splits the 10 examples into different groups:
 - **Left Branch:** 4 examples (4 Cats, 0 Dogs)
 - **Right Branch:** 6 examples (1 Cat, 5 Dogs)
 - This split is much "purer." The algorithm calculates the information gain and finds **Information Gain = 0.61**. This is much better!
3. **Try Another Threshold (e.g., `weight <= 13`):**

- This split results in **Information Gain = 0.40**.

The algorithm will test many possible thresholds to find the one that **maximizes information gain**. In this case, `weight <= 9` (with a gain of 0.61) is the best split for this feature.

The Full Splitting Algorithm

At any given node, the algorithm performs the following steps:

1. **For each categorical feature** (e.g., `Ear Shape`): Calculate its information gain.
 2. **For each continuous feature** (e.g., `weight`):
 - a. Find the **best possible threshold** by testing many different values and calculating the information gain for each.
 - b. The "score" for this feature is the *highest* information gain it can achieve (e.g., 0.61 for `weight`).
 3. **Final Decision:** Compare the best scores from all features. The algorithm splits on the feature that provided the **single highest information gain** overall.
 - (e.g., If `Ear Shape` gave 0.28 gain and `weight` gave 0.61, the algorithm would choose to split the node on `weight <= 9`).
-

Pro-Tip: How are thresholds chosen to be tested?

How does the algorithm test "many different values" efficiently? A common convention is:

1. **Sort** all examples at the node based on the continuous feature's value (e.g., sort all 10 animals by `weight`).
2. Test the **midpoint** value between each consecutive pair of examples.
3. (This way, if you have 10 examples, you only need to test 9 different thresholds to find the best possible split.)

Regression Trees (Optional) 🌳

Decision trees can be generalized to solve **regression problems**, where the goal is to predict a continuous numerical value (like a price or, in this case, an animal's **weight**) instead of a discrete class (like "Cat" or "Not Cat").

1. How Regression Trees Make Predictions

The structure of the tree (splitting on features like "Ear Shape" or "Face Shape") is built in the same way. The only difference is the value stored in the **leaf nodes**.

- **Classification Tree Leaf:** Predicts the **majority class** (e.g., "Cat") of the training examples that fall into that leaf.
- **Regression Tree Leaf:** Predicts the **average value** of the target y (e.g., the average weight) of all the training examples that fall into that leaf.

Example:

If 4 animals (with weights 7.2, 7.6, 8.4, 10.2) end up in a leaf, the prediction for that leaf becomes their average: **8.35 lbs**.

2. How Regression Trees are Trained: Reducing Variance

To build a regression tree, we need a different metric to measure "impurity." For classification, we minimized **entropy**. For regression, we minimize **variance**.

- **Entropy (Classification):** Measures the *impurity* of classes (how mixed "Cat" and "Not Cat" are).
 - **Variance (Regression):** Measures the *impurity* of numbers (how "spread out" the weights are).
 - A set of numbers like [8.8, 15, 11, 18, 20] has **high variance**.
 - A set of numbers like [7.2, 7.6, 8.4, 10.2] has **low variance**.
 - The goal is to create splits that result in nodes with the lowest possible variance.
-

Finding the Best Split 📊

The algorithm's goal is to find the feature split that results in the **largest reduction in variance**. This is the direct equivalent of seeking the highest "information gain" in a classification tree.

The process at each node is:

1. **Calculate Parent Variance:** First, calculate the variance of all y values (weights) in the

current (parent) node.

- *Example:* Variance of all 10 animals = **20.51**

2. Test Each Feature Split:

For each possible feature (e.g., "Ear Shape"):

- **a.** Split the data and calculate the variance for the left and right child nodes (e.g., $\text{Variance_left} = 1.47, \text{Variance_right} = 21.87$).
- **b.** Calculate the **weighted average variance** of the children.
 - $\text{avg_variance} = (w_{\text{left}} * \text{Variance_left}) + (w_{\text{right}} * \text{Variance_right})$
 - (e.g., $(5/10 * 1.47) + (5/10 * 21.87) = 11.67$)

3. Calculate Variance Reduction:

- $\text{Variance Reduction} = \text{Parent Variance} - \text{Weighted Average Variance}$
- *Example (Ear Shape):* $20.51 - 11.67 = 8.84$
- *Example (Face Shape):* $20.51 - 19.87 = 0.64$
- *Example (Whiskers):* $20.51 - 14.29 = 6.22$

4. Decision:

- The "Ear Shape" split gives the **highest variance reduction** (8.84).
- Therefore, the algorithm chooses "Ear Shape" as the feature for this split.

This recursive process continues until a stopping criterion (like `max_depth` or minimum examples) is met.